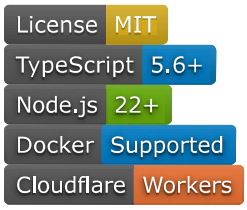
Puppeteer

Browser automation using Puppeteer, with support for local, Docker, and Cloudflare Workers deployments.

GitHub 3

Puppeteer MCP Server

A comprehensive Model Context Protocol (MCP) server that provides browser automation capabilities using Puppeteer. This server enables Large Language Models (LLMs) to interact with web pages, take screenshots, and execute JavaScript in both local and cloud environments.



🚀 Features

Core Capabilities

- **Browser Automation**: Navigate, click, fill forms, and interact with web pages
- **Screenshot Capture**: Take full-page or element-specific screenshots
- **JavaScript Execution**: Run custom scripts in browser context
- **Console Monitoring**: Capture and access browser console logs
- **Resource Management**: Store and retrieve screenshots and logs
- **Security Controls**: Configurable dangerous argument filtering

Deployment Options

- **Local Development**: Direct Puppeteer integration with Chromium
- **Docker Container**: Containerized deployment with ARM64 support

Cloudflare Workers: Cloud deployment using external browser services

📦 Installation & Usage

Option 1: NPX (Recommended for Local Development)

```
npx -y @modelcontextprotocol/server-puppeteer
```

Option 2: Docker

```
docker run -i --rm --init -e DOCKER_CONTAINER=true puppeteer-mcp
```

Option 3: Build from Source

```
git clone https://github.com/code-craka/puppeteer-mcp.git
cd puppeteer-mcp
npm install
npm run build
node dist/index.js
```

🛠️ Available Tools

Navigation & Interaction

- **`puppeteer_navigate`** - Navigate to URLs with optional launch options
- **`puppeteer_click`** - Click elements using CSS selectors
- **`puppeteer_hover`** - Hover over elements
- **`puppeteer_fill`** - Fill input fields and forms
- **`puppeteer_select`** - Select dropdown options

Content & Automation

- **`puppeteer_screenshot`** - Capture page or element screenshots
- **`puppeteer_evaluate`** - Execute JavaScript in browser context

Resources

- **`console://logs`** - Access browser console output
- **`screenshot://<name>`** - Retrieve captured screenshots

🔧 Configuration

Claude Desktop Configuration

Add to your `claude_desktop_config.json` :

```
{
  "mcpServers": {
    "puppeteer": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"]
    }
  }
}
```

Docker Configuration

```
{
  "mcpServers": {
    "puppeteer": {
      "command": "docker",
      "args": ["run", "-i", "--rm", "--init", "-e", "DOCKER_CONTAINER=true"
    }
  }
}
```

Environment Variables

- **`PUPPETEER_LAUNCH_OPTIONS`** - JSON-encoded browser launch options
- **`ALLOW_DANGEROUS`** - Enable dangerous browser arguments (default: false)
- **`DOCKER_CONTAINER`** - Container detection flag

Browser Launch Options

```
{
  "mcpServers": {
    "puppeteer": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"],
      "env": {
        "PUPPETEER_LAUNCH_OPTIONS": "{\"headless\": false, \"args\": [\"--r
        "ALLOW_DANGEROUS": "true"
      }
    }
```

```
  }
}
```

📐 Development

Local Development

```
# Clone the repository
git clone https://github.com/code-craka/puppeteer-mcp.git
cd puppeteer-mcp

# Install dependencies
npm install

# Development with watch mode
npm run watch

# Build for production
npm run build
```

Docker Development

```
# Build Docker image
docker build -t puppeteer-mcp .

# Run with environment variables
docker run -i --rm --init \
  -e DOCKER_CONTAINER=true \
  -e PUPPETEER_LAUNCH_OPTIONS='{"headless":true}' \
  puppeteer-mcp
```

☁️ Cloudflare Workers Deployment

🚀 Live Production Deployment

Live URL: `https://puppeteer.techsci.dev`

Our Puppeteer MCP server is successfully deployed on Cloudflare Workers with Browserless.io integration.

✅ Deployment Status

Status: Live and operational

Browser Service: Browserless.io connected

API Calls: Screenshots working ✅

Tools Available: 6 browser automation tools

Response Time: ~1-2 seconds average

🛠️ Deploy Your Own

```
cd cloudflare-worker
npm install                # Installs Wrangler v4.22.0 + secure dependencie
npx wrangler login
echo "YOUR_BROWSERLESS_TOKEN" | npx wrangler secret put BROWSERLESS_TOKEN
npm run build             # Compile TypeScript to dist/index.js
npm run deploy           # Deploy using latest Wrangler v4.22.0
```

🧪 Test the Live Deployment

```
# Test tools listing
curl -X POST https://puppeteer.techsci.dev \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"tools/list","id":"test"}'

# Test screenshot capture
curl -X POST https://puppeteer.techsci.dev \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"tools/call","params":{"name":"browser_scre
```

💰 Production Costs

Cloudflare Workers: 100,000 requests/day free

Browserless.io: ~$0.0025 per second of browser usage

Monthly estimate: $10-50 for moderate usage

See Cloudflare Worker README for detailed setup instructions.

🔐 Security

Default Security Measures

Dangerous browser arguments are filtered by default

Configurable security levels via environment variables

Headless mode in Docker containers

Scoped permissions for different deployment environments

Latest Updates: All dependencies updated, 0 security vulnerabilities

Secure Build: esbuild v0.25.0+ and Wrangler v4.22.0 with latest patches

Dangerous Arguments (Filtered by Default)

```
--no-sandbox
--disable-setuid-sandbox
--single-process
--disable-web-security
--ignore-certificate-errors
```

Override with `ALLOW_DANGEROUS=true` environment variable.

📋 Examples

Basic Screenshot

```
{
  "name": "puppeteer_screenshot",
  "arguments": {
    "name": "example-page",
    "width": 1280,
    "height": 720
  }
}
```

Navigate and Interact

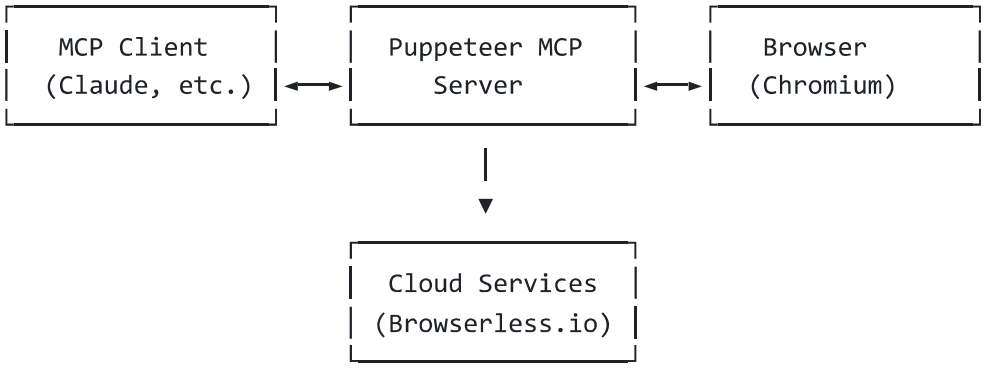
```
{
  "name": "puppeteer_navigate",
  "arguments": {
    "url": "https://example.com",
    "launchOptions": {
      "headless": false
    }
  }
}
```

Execute JavaScript

```
{
  "name": "puppeteer_evaluate",
```

```
  "arguments": {
    "script": "return document.title;"
  }
}
```

📊 Architecture



🤝 Contributing

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/amazing-feature`
3. Commit your changes: `git commit -m 'Add amazing feature'`
4. Push to the branch: `git push origin feature/amazing-feature`
5. Open a Pull Request

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

🙏 Acknowledgments

Model Context Protocol - The protocol that makes this possible

Puppeteer - Headless Chrome Node.js API

Anthropic - Original MCP server implementation

Browserless.io - Cloud browser automation service

🔗 Related Projects

MCP SDK - Model Context Protocol SDK

Claude Desktop - AI assistant with MCP support

Puppeteer - Browser automation library

📞 Support

Create an issue for bug reports

Join discussions in GitHub Discussions

Check the CLAUDE.md for development guidance

🏷️ Version History

v1.0.0 - Initial release with local Puppeteer support

v1.1.0 - Added Docker support and ARM64 compatibility

v1.2.0 - Cloudflare Workers adaptation with external browser services

v1.3.0 - ✅ **Live Production Deployment** - Successfully deployed on Cloudflare Workers with Browserless.io integration, tested and confirmed working

v1.4.0 - 🔒 **Security & Performance Update** - Updated Wrangler to v4.22.0, fixed esbuild vulnerabilities, added observability logs, resolved all npm audit issues (0 vulnerabilities)

Author: Sayem Abdullah Rihan
License: MIT
Repository: github.com/code-craka/puppeteer-mcp

Related Servers

Bright Data sponsor

Discover, extract, and interact with the web - one interface powering automated access across the public internet.

Open Crawler MCP Server

A web crawler and text extractor with robots.txt compliance, rate limiting, and page size protection.

Playwright MCP

Control a browser for automation and web scraping tasks using Playwright.

yt-dlp

Download video and audio content from various websites like YouTube, Facebook, and Tiktok using yt-dlp.

Google Flights

An MCP server to interact with Google Flights data for finding flight information.

Web Scout

A server for web scraping, searching, and analysis using multiple engines and APIs.

News MCP Server

Real-time news aggregation from AP, BBC, NPR, Hacker News, and Google News

MCP Image Downloader

A server for downloading and optimizing images from the web.

Secure Fetch

Secure fetch to prevent access to local resources

scraping-api-marketplace

Real-time product data from Amazon, eBay, Walmart, Kaufland and many others — directly inside your AI assistant

MCP Web Research Server

A server for web research that brings real-time information into AI models and researches any topic.

contact@mcpservers.org